ARQUITETURA DE SOFTWARE E SOLUÇÕES CLOUD · PUCPR

CloudLog — Documentação de Arquitetura

Documentação completa baseada no template arc42, com C4 Model (níveis 1 a 4), UML, Diagrama de Entidades e Relacionamentos e Software Architecture Canvas.

INTEGRANTES	Camilla Augusta Uber João Davi Laura Guillarducci
PROFESSOR	Manoel Valerio da Silveira Neto

VERSÃO	1.0 — documentação completa (evolução da versão parcial 0.1)
DATA	14/10/2026
REPOSITÓRIO	https://github.com/laducci/cloudlog-pjbl
APLICAÇÃO	https://polite-mushroom-0b05fa610.3.azurestaticapps.net
TEMPLATE	arc42 (arc42.org), seções 1 a 11 e glossário

Sumário

1	Introdução e Objetivos	3
2	Restrições Arquiteturais	4
3	Contexto e Escopo — C4 Nível 1	5
4	Estratégia de Solução, estilos arquiteturais e Software Architecture Canvas	7
5	Visão de Blocos de Construção — C4 Níveis 2, 3 e 4 (UML Classes e Componentes) e DER	9
6	Visão de Execução — C4 Nível 4 (UML Sequência) e eventos	20
7	Visão de Implantação	25
8	Conceitos Transversais e Testes de Arquitetura	27
9	Decisões Arquiteturais (ADRs)	29
10	Requisitos de Qualidade	30
11	Riscos e Débitos Técnicos	31
12	Glossário	32

Como ler este documento

O documento segue a estrutura do template oficial arc42. Cada seção começa com um quadro cinza que resume o propósito da seção no template, seguido do conteúdo do CloudLog. Os diagramas aparecem dentro da seção correspondente:

Artefato pedido	Onde está
C4 Nível 1 — Contexto	Seção 3.2
C4 Nível 2 — Contêineres	Seção 5.1
C4 Nível 3 — Componentes	Seção 5.5
C4 Nível 4 — UML Classes e UML Componentes	Seção 5.7
C4 Nível 4 — UML Sequência	Seção 6
Diagrama de Entidades e Relacionamentos	Seção 5.8
Software Architecture Canvas	Seção 4.3
Clean Architecture e Vertical Slice (reflete o código)	Seção 5.6 (pacotes), 5.7 (classes)
Testes unitários de arquitetura (reflete o código)	Seção 8.6
Serverless · Microfrontend · BFF + Microservices + Database per Service · API Gateway · EDA	Seção 4.1 (visão geral), 5.2 a 5.4, 6 e 7
Visão de implantação (UML Deployment)	Seção 7

Legenda de situação

implementado existe no código entregue no repositório parcial existe em parte planejado faz parte da arquitetura-alvo documentada e ainda não foi construído. Nos diagramas C4, elementos em azul claro são da arquitetura-alvo.

SEÇÃO 1

1 Introdução e Objetivos

> arc42: descreve os requisitos relevantes e as forças que direcionam a arquitetura — objetivos de negócio, principais objetivos de qualidade e partes interessadas.

O **CloudLog** é uma plataforma em nuvem para a gestão de operações de transporte rodoviário de cargas. Ela centraliza o planejamento de rotas e entregas, o rastreamento da frota em tempo real, a comprovação digital das entregas, a emissão de alertas de atraso ou desvio e a consolidação de indicadores operacionais.

O problema que motiva a solução é a baixa visibilidade da operação em tempo real, somada a falhas na gestão de rotas, a atrasos na comunicação entre a central e os motoristas e à dependência de comprovantes em papel.

1.1 Visão geral dos requisitos

Objetivos de negócio

- Reduzir os custos operacionais de transporte em até 15%.
- Reduzir o tempo de atraso nas entregas em 25%.
- Digitalizar 100% da comprovação de entregas.
- Aumentar a rastreabilidade da frota e a velocidade de resposta da central.

Requisitos funcionais

ID	Requisito	Descrição	Usuário principal	Situação
RF-01	Autenticação e Controle de Acesso	Login seguro com MFA e controle de acesso por perfil (RBAC).	Todos	planejado
RF-02	Rastreamento em Tempo Real	Geolocalização dos veículos por telemetria/GPS.	Gerente de Logística	planejado
RF-03	Gestão de Rotas e Entregas	Criação, edição, pesquisa, exclusão e atribuição de entregas.	Gerente de Logística	implementado entregas
RF-04	Comprovação Digital de Entrega	Fotos e assinatura digital pelo aplicativo do motorista.	Motorista	planejado
RF-05	Alertas de Atraso/Desvio	Alertas automáticos para a central em caso de atraso ou desvio.	SAC / Gerente	parcial atraso
RF-06	Dashboard de Desempenho	Indicadores de tempo de rota, consumo e eficiência.	Gerente de Logística	parcial tela

1.2 Objetivos de qualidade

Prioridade	Objetivo de qualidade	Motivação
1	Disponibilidade	A central e os motoristas dependem da plataforma durante toda a execução das rotas.
2	Segurança	Dados pessoais, localização e evidências de entrega exigem autenticação forte e autorização por perfil.
3	Escalabilidade e custo	Telemetria e uso variam muito ao longo do dia; a solução deve escalar sob demanda e custar pouco ociosa.
4	Manutenibilidade e testabilidade	Equipe pequena e evolução contínua: regras isoladas, fatias independentes e regras de arquitetura verificadas por teste.
5	Usabilidade	O motorista precisa atualizar entregas e registrar comprovantes com poucos passos.

1.3 Partes interessadas

Parte interessada	Expectativa
Carlos — Gerente de Logística	Visibilidade da frota, criação de rotas e entregas, alertas e indicadores para reduzir custos.
Marcos — Motorista de Frota	Aplicativo simples para ver rotas, atualizar status e enviar comprovantes.
Fernanda — Atendente de SAC	Consulta rápida de status, localização e atrasos para responder clientes.
Equipe de desenvolvimento	Código modular, testável e com fronteiras claras entre fatias e camadas.
Operação de TI e gestão	Serviços gerenciados, observabilidade, custo previsível e as metas de negócio da Seção 1.1.

SEÇÃO 2

2 Restrições Arquiteturais

> arc42: lista as restrições técnicas, organizacionais e as convenções que limitam a liberdade de decisão da equipe.

Na versão parcial a solução era independente de provedor. Nesta versão as restrições foram fixadas a partir do que foi efetivamente construído nas atividades da disciplina.

2.1 Restrições técnicas

ID	Restrição	Impacto na arquitetura
RT-01	Nuvem Microsoft Azure com a assinatura **Azure for Students**.	Prioriza serviços com camada gratuita ou de consumo (Static Web Apps, Functions Consumption, Service Bus Basic).
RT-02	Backend serverless em **Azure Functions** (Node.js, modelo de programação v4).	Código sem estado, inicialização rápida e reuso de conexões entre execuções.
RT-03	Frontend publicado no **Azure Static Web Apps**, com funções gerenciadas.	As funções gerenciadas aceitam apenas gatilhos HTTP; o consumidor de fila roda numa Function App dedicada.
RT-04	Persistência do Serviço de Entregas no **MongoDB Atlas** (Student Pack, cluster M0).	Modelo de documentos; o cluster gratuito pausa após inatividade.
RT-05	Integração com GPS/IoT para o rastreamento (RF-02).	Ingestão de alto volume separada do fluxo transacional.
RT-06	MFA e RBAC (RF-01).	Autenticação delegada a um provedor de identidade e autorização validada na borda.
RT-07	Fotos e assinaturas (RF-04).	Arquivos em Object Storage, nunca no banco transacional.

2.2 Restrições organizacionais

ID	Restrição	Descrição
RO-01	Equipe	Grupo do PJBL: Camilla Augusta Uber, João Davi e Laura Guillarducci.
RO-02	Prazo e escopo acadêmico	Entregas incrementais por atividade; parte da arquitetura é documentada como alvo e construída aos poucos.
RO-03	Documentação	Template arc42 com 11 seções, C4 Model níveis 1 a 4, UML, DER e Software Architecture Canvas, entregue em PDF.
RO-04	Uso de IA generativa	Permitido nas atividades; os prompts usados para alterar o código ficam registrados no repositório.
RO-05	Código e versionamento	Repositório público no GitHub; deploy contínuo pelo GitHub Actions na branch main.

2.3 Convenções

ID	Convenção
C-01	Requisitos identificados como RF-XX; decisões como ADR-XX; riscos como R-XX.
C-02	Código organizado por funcionalidade em `api/src/features/<fatia>` (Vertical Slice).
C-03	Nomes de classes e eventos em inglês no código; documentação e mensagens ao usuário em português.
C-04	Diagramas como código (PlantUML, notação C4-PlantUML e UML) versionados em `docs/arquitetura/diagramas`.
C-05	Toda mudança no backend precisa passar nos testes unitários e de arquitetura (`npm test`).

SEÇÃO 3

3 Contexto e Escopo

> arc42: delimita o sistema em relação aos usuários e sistemas externos, do ponto de vista de negócio e técnico.

3.1 Contexto de negócio

Elemento externo	Entradas para o CloudLog	Saídas do CloudLog
Gerente de Logística	Rotas, entregas, atribuições e consultas.	Status, localização, alertas e indicadores.
Atendente de SAC	Consultas sobre entregas e ocorrências.	Status, localização e informações de atraso.
Motorista de Frota	Atualizações de status, fotos e assinatura.	Rotas do dia, dados das entregas e confirmações.
Dispositivos GPS / IoT	Posições e telemetria dos veículos.	—
Provedor de Identidade	Tokens e identidade do usuário (MFA).	Pedidos de autenticação.
Serviço de Notificações	Confirmação de envio.	Alertas por push, e-mail ou SMS.

3.2 C4 Model — Nível 1: Contexto do Sistema

O diagrama da Figura 1 mostra o CloudLog como uma caixa-preta, com as três personas e os três sistemas externos. Setas tracejadas em roxo indicam comunicação assíncrona.

3.3 Contexto técnico

Interface	Canal / protocolo	Formato	Situação
Frontend → Serviço de Entregas	HTTPS, rotas `/api/deliveries` e `/api/hello`	JSON UTF-8	implementado
Serviço de Entregas → MongoDB Atlas	MongoDB driver sobre TLS	BSON	implementado
Serviço de Entregas → Service Bus	AMQP sobre TLS (fila `cloudlog-delivery-events`)	JSON (evento versionado)	implementado
Service Bus → Serviço de Alertas	ServiceBusQueueTrigger	JSON	implementado
Clientes → API Gateway → BFFs	HTTPS com token JWT	JSON	planejado
Shell → Microfrontends	Module Federation (remoteEntry.js)	JavaScript	planejado
GPS/IoT → IoT Hub	MQTT	JSON	planejado
CloudLog → Provedor de Identidade	OpenID Connect / OAuth 2.0	JWT	planejado

SEÇÃO 3.2 · C4 NÍVEL 1

C4 Nível 1 — Contexto do Sistema

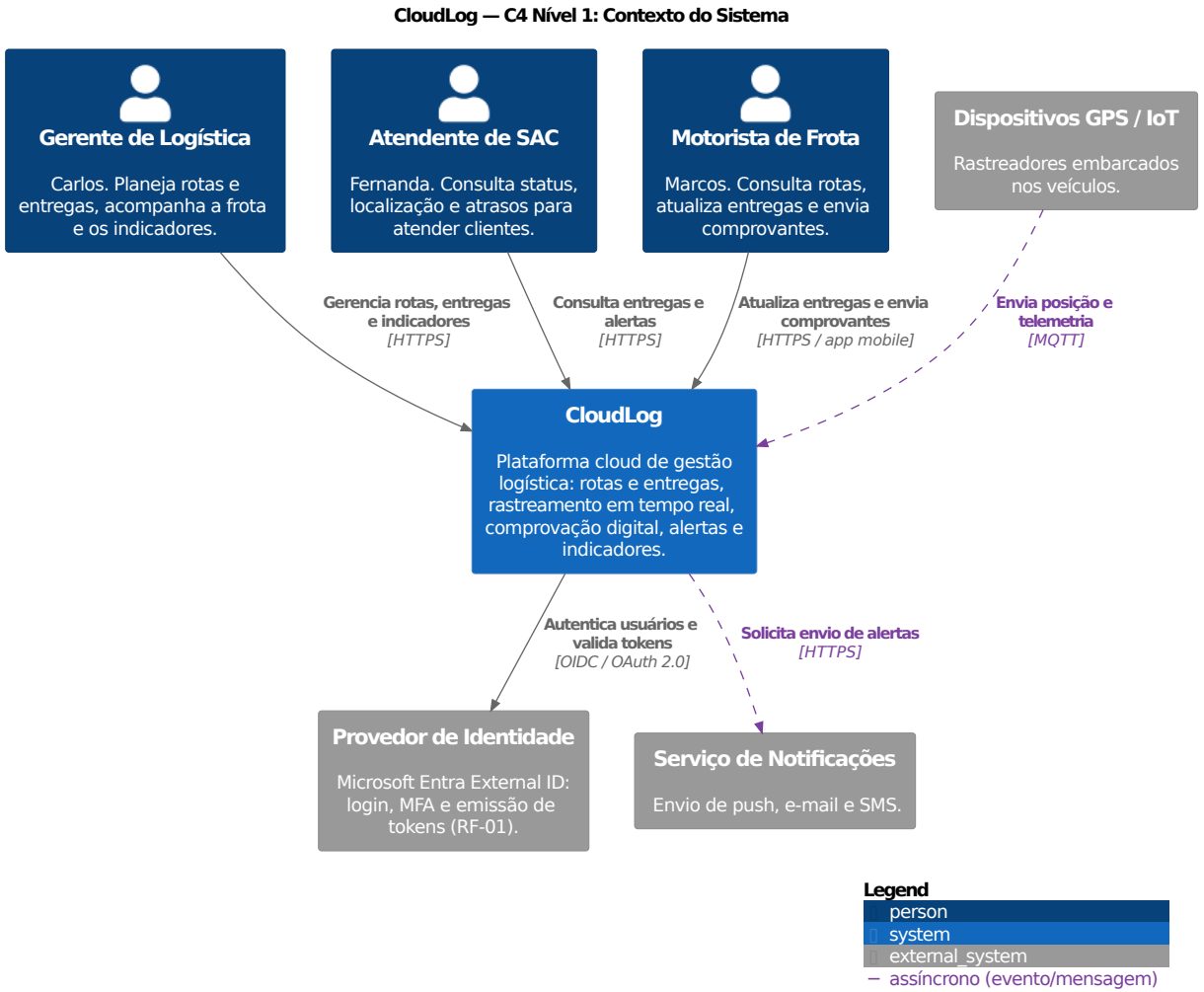


Figura 1 — Contexto do CloudLog: pessoas, sistema e sistemas externos. Fonte: `docs/arquitetura/diagramas/c4-1-contexto.puml`

SEÇÃO 4

4 Estratégia de Solução

> arc42: resume as decisões fundamentais e as estratégias que moldam a arquitetura — tecnologia, decomposição de alto nível e abordagens para atingir os objetivos de qualidade.

A estratégia combina **serverless** na execução, **microsserviços com banco por serviço** na decomposição do backend, **API Gateway + BFF** na borda, **microfrontends** no cliente web e **eventos** para integrar os serviços sem acoplamento temporal. Dentro de cada serviço, o código segue **Clean Architecture** organizada em **fatias verticais**, e as regras de dependência são protegidas por **testes unitários de arquitetura**.

O Serviço de Entregas foi construído primeiro porque atende o RF-03, que é a base dos demais fluxos. Ele já aplica todos os padrões internos (Clean Architecture, Vertical Slice, SOLID, EDA e testes de arquitetura) e serve de modelo para os próximos serviços.

4.1 Estilos arquiteturais adotados

Estilo arquitetural	Como é aplicado no CloudLog	Onde	Situação
Serverless	Todo o backend roda em Azure Functions no plano de consumo (escala a zero, cobrança por execução). Banco, mensageria e hospedagem são serviços gerenciados (MongoDB Atlas, Service Bus, Static Web Apps).	`api/`, Seção 7	implementado
Microfrontend	Um Shell React (host) carrega em tempo de execução três microfrontends independentes — Entregas, Operação e Rastreamento — via Module Federation. Cada MFE tem deploy próprio.	Seção 5.2	parcial telas existem numa SPA
Backend for Frontend (BFF)	BFF Web (gerente e SAC) agrega entregas, posições e indicadores numa única resposta; BFF Mobile entrega payload enxuto e sincronização offline ao motorista.	Seções 5.3 e 6.4	planejado
Microservices + Database per Service	Cinco serviços por capacidade de negócio (Entregas, Telemetria, Comprovantes, Alertas, Indicadores). Cada serviço é dono exclusivo do seu banco; dados de outro serviço só por API ou evento.	Seções 5.1, 5.4 e 5.8	parcial Entregas + Alertas
API Gateway	Azure API Management como porta única: valida JWT e perfil (RBAC), aplica rate limit e CORS e roteia `/web/*` e `/mobile/*` para os BFFs. Hoje o roteamento `/api` é feito pelo próprio Static Web Apps.	Seções 5.3 e 7	parcial
Arquitetura Orientada a Eventos (EDA)	Inserir, alterar e excluir entregas publicam os eventos de domínio `DeliveryCreated`, `DeliveryUpdated` e `DeliveryDeleted` no Azure Service Bus. O Serviço de Alertas consome os eventos e gera o alerta de atraso (RF-05).	Seções 6.2 e 8.4	implementado
Clean Architecture + Vertical Slice	Backend organizado por fatias (`features/<funcionalidade>`), cada uma com function, handler e caso de uso. Domínio puro no centro; adaptadores na borda; Composition Root faz a injeção de dependência.	Seções 5.6 e 5.7	implementado
Testes unitários de arquitetura	13 testes com `node:test` leem as dependências de cada arquivo de `api/src` e falham se a regra de dependência ou o isolamento das fatias for violado. Rodam no GitHub Actions.	Seção 8.6	implementado

SEÇÃO 4

4.2 Abordagens para os objetivos de qualidade

Objetivo de qualidade	Abordagem da solução
Disponibilidade	Serviços gerenciados com SLA, funções sem estado, desacoplamento por fila (uma falha no consumidor não derruba a API).
Segurança	Identidade externa com MFA, validação de token no API Gateway, segredos em configurações da plataforma, TLS em todas as conexões.
Escalabilidade e custo	Serverless por consumo, ingestão de telemetria isolada, banco por serviço escalado de forma independente.
Manutenibilidade	Vertical Slice + Clean Architecture, SOLID, portas e adaptadores, testes de arquitetura no CI.
Usabilidade	BFF Mobile com payload enxuto, microfrontends focados em cada perfil.

4.3 Software Architecture Canvas

O canvas resume em uma página os principais direcionadores da arquitetura do CloudLog.

PROPOSTA DE VALOR

Centralizar a operação logística em nuvem: rastrear a frota em tempo real, reduzir atrasos com alertas automáticos e eliminar comprovantes em papel.

PARTES INTERESSADAS

- Gerente de Logística (Carlos)
- Atendente de SAC (Fernanda)
- Motorista (Marcos)
- Operação de TI e equipe de desenvolvimento

FUNCIONALIDADES-CHAVE

- RF-01 Login MFA + RBAC
- RF-02 Rastreamento
- RF-03 Rotas e entregas
- RF-04 Comprovação digital
- RF-05 Alertas
- RF-06 Indicadores

ATRIBUTOS DE QUALIDADE

- Disponibilidade
- Segurança
- Escalabilidade e custo
- Manutenibilidade e testabilidade
- Usabilidade

RESTRIÇÕES DE NEGÓCIO

- Reduzir custos em 15%
- Reduzir atrasos em 25%
- 100% comprovação digital
- Equipe de 3 alunos e prazo acadêmico

RESTRIÇÕES TÉCNICAS

- Azure for Students
- Azure Functions (Node.js)
- Static Web Apps só HTTP
- MongoDB Atlas M0
- GPS/IoT via MQTT

DECISÕES ARQUITETURAIS

- Serverless (ADR-03) e microsserviços com banco por serviço (ADR-02)
- API Gateway + BFF por canal (ADR-07, ADR-08)
- Microfrontends com Module Federation (ADR-09)
- Eventos no Service Bus (ADR-06) · Clean Architecture + Vertical Slice (ADR-10)

COMPONENTES PRINCIPAIS

- Shell Web + MFEs Entregas, Operação e Rastreamento; App do Motorista
- API Gateway, BFF Web, BFF Mobile
- Serviços de Entregas, Telemetria, Comprovantes, Alertas e Indicadores
- Barramento de eventos e IoT Hub

TECNOLOGIAS

React, Vite, Module Federation, React Native · Azure Static Web Apps, Azure Functions, API Management, Service Bus, IoT Hub, Cosmos DB, Blob/Table Storage, Azure SQL, Application Insights · MongoDB Atlas · GitHub Actions · node:test

RISCOS

- Perda de evento sem outbox
- Cluster M0 pausado
- Rotas ainda sem autenticação
- Complexidade de microsserviços

QUESTÕES EM ABERTO

- Custo do APIM e IoT Hub fora da conta estudante
- Retenção da telemetria
- Quando migrar para Module Federation

SEÇÃO 5

5 Visão de Blocos de Construção

> arc42: mostra a decomposição estática do sistema em blocos (contêineres, componentes, classes), do nível mais geral (caixa-branca do sistema) até o detalhe necessário.

5.1 Nível 1 da decomposição — C4 Nível 2: Contêineres

A Figura 2 abre o CloudLog e mostra todos os contêineres da arquitetura-alvo, reunindo os estilos pedidos: microfrontends no cliente web, API Gateway e BFFs na borda, microsserviços serverless com banco exclusivo e um barramento de eventos.

Contêiner	Tecnologia	Responsabilidade	Situação
Shell Web	React + Vite (host Module Federation)	Login, layout, menu e carregamento dos MFEs.	parcial
MFE Entregas	React (remote)	Pesquisa, inserção, alteração e exclusão de entregas.	parcial
MFE Operação	React (remote)	Indicadores, telemetria resumida e alertas.	parcial
MFE Rastreamento	React (remote)	Mapa da frota em tempo real.	planejado
App do Motorista	React Native	Entregas do dia e comprovação digital.	planejado
API Gateway	Azure API Management	Autenticação na borda, rate limit e roteamento.	planejado
BFF Web / BFF Mobile	Azure Functions	Agregação e adaptação de respostas por canal.	planejado
Serviço de Entregas	Azure Functions + MongoDB Atlas	CRUD de entregas e publicação de eventos.	implementado
Serviço de Alertas	Azure Functions (ServiceBusQueueTrigger)	Consome eventos e gera alertas.	implementado
Serviços de Telemetria, Comprovantes e Indicadores	Azure Functions + Cosmos DB / Blob / Azure SQL	RF-02, RF-04 e RF-06.	planejado
Barramento de Eventos	Azure Service Bus	Transporte de eventos de domínio.	implementado

5.2 Microfrontends

O frontend web é dividido por domínio de negócio. O Shell é o único ponto que o usuário acessa; ele faz o login e carrega cada microfrontend sob demanda. Cada MFE é publicado de forma independente e só conversa com o BFF Web.

Microfrontend	Rota no Shell	Expõe (Module Federation)	Consome	Equipe dona
Shell (host)	/	—	remoteEntry.js dos MFEs	Plataforma
MFE Entregas	/entregas	`./DeliveriesPage`	BFF Web `/web/deliveries`	Entregas
MFE Operação	/operacao	`./OperationsDashboard`	BFF Web `/web/dashboard`	Indicadores
MFE Rastreamento	/rastreamento	`./FleetMap`	BFF Web `/web/positions`	Telemetria

Hoje as telas Operação e Entregas já existem, mas no mesmo bundle React (SPA única). A separação em remotes com Module Federation está registrada como débito técnico (Seção 11).

SEÇÃO 5

5.3 API Gateway e Backend for Frontend

O API Gateway é a única porta pública da API. Ele aplica as políticas transversais (autenticação, autorização por perfil, limites de uso e CORS) e encaminha cada canal para o seu BFF. Os BFFs não têm regras de negócio: apenas agregam e adaptam as respostas dos microsserviços para a tela que as consome.

Rota pública (API Gateway)	Destino	Políticas na borda	Agregação no BFF
`GET /web/dashboard`	BFF Web	JWT válido; perfil Gerente ou SAC; rate limit 60/min	Entregas + Telemetria + Indicadores (Sequência 4)
`GET/POST/PUT/DELETE /web/deliveries`	BFF Web → Serviço de Entregas	JWT; POST/PUT/DELETE só Gerente	Repasse com adaptação de payload
`GET /mobile/today`	BFF Mobile	JWT; perfil Motorista	Somente as entregas do motorista, campos mínimos
`POST /mobile/proofs`	BFF Mobile → Serviço de Comprovantes	JWT; tamanho máximo de upload	Gera URL SAS para envio direto ao Blob

5.4 Microsserviços e Database per Service

Cada microsserviço é dono exclusivo do seu armazenamento. Nenhum serviço lê o banco de outro: dados de outro contexto chegam por chamada HTTP (síncrona) ou por evento (assíncrona).

Microsserviço	Capacidade de negócio	Banco exclusivo	Publica	Consome
Entregas implementado	Rotas e entregas (RF-03)	MongoDB Atlas — `cloudlog.deliveries`	DeliveryCreated, DeliveryUpdated, DeliveryDeleted	—
Alertas implementado	Alertas de atraso e desvio (RF-05)	Table Storage — alertas	AlertRaised	DeliveryCreated, DeliveryUpdated, RouteDeviationDetected
Telemetria planejado	Rastreamento (RF-02)	Cosmos DB — posições	PositionReceived, RouteDeviationDetected	Mensagens do IoT Hub
Comprovantes planejado	Comprovação digital (RF-04)	Blob Storage + metadados	ProofSubmitted	—
Indicadores planejado	Dashboards (RF-06)	Azure SQL — modelo analítico	—	Todos os eventos (projeções de leitura)

SEÇÃO 5.1 · C4 NÍVEL 2

C4 Nível 2 — Contêineres

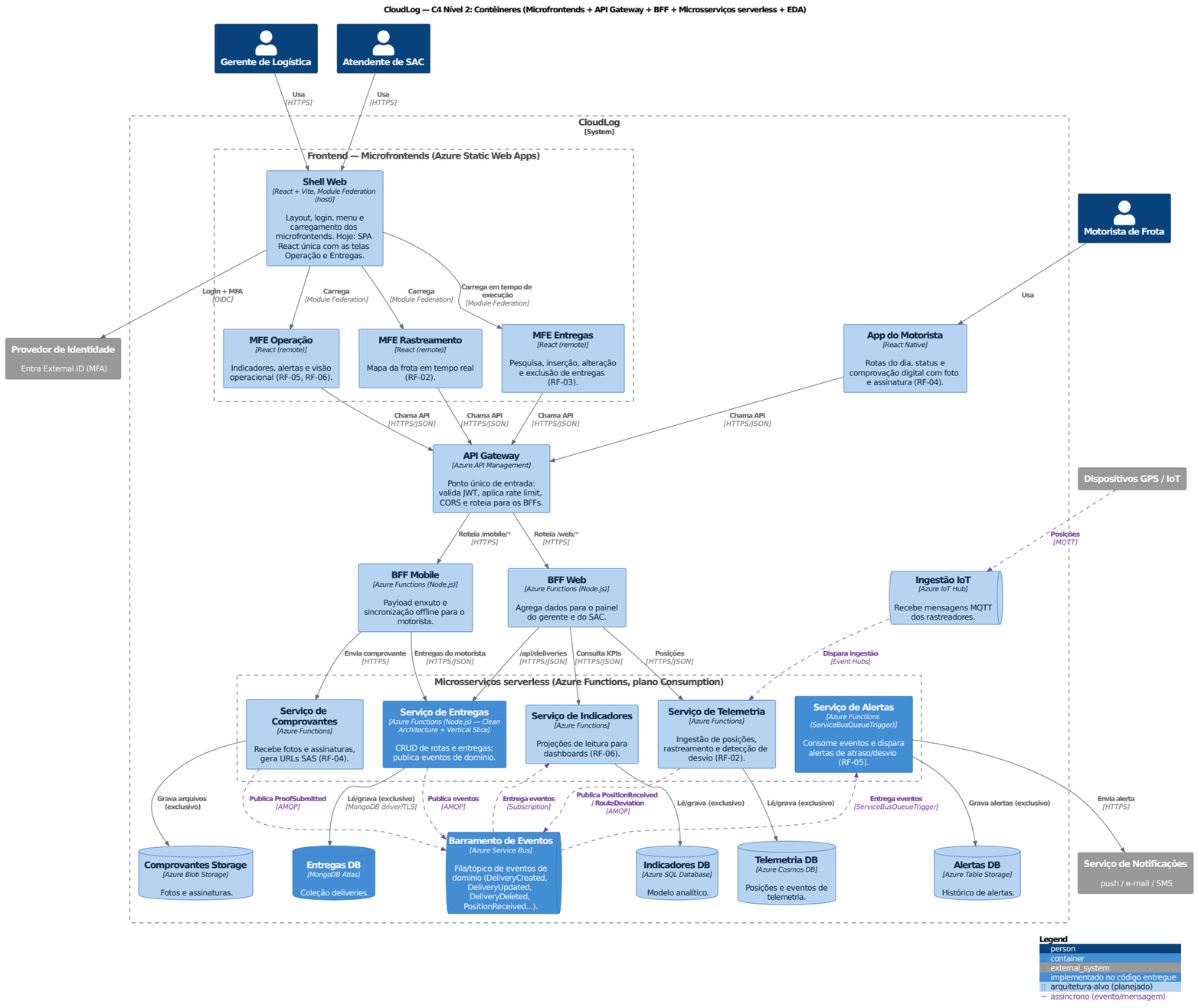


Figura 2 — Contêineres do CloudLog. Azul escuro: implementado; azul claro: arquitetura-alvo; roxo tracejado: assíncrono. Fonte: `docs/arquitetura/diagramas/c4-2-conteineres.puml`

SEÇÃO 5

5.5 Nível 2 da decomposição — C4 Nível 3: Componentes do Serviço de Entregas

A Figura 3 abre o contêiner Serviço de Entregas e mostra seus componentes exatamente como estão em `api/src`. As funções HTTP delegam aos handlers, que executam os casos de uso. Os casos de uso dependem apenas do domínio e das portas; os adaptadores de infraestrutura implementam as portas e são instanciados pelo Composition Root. O consumidor `processDeliveryEvent` está no mesmo repositório, mas é implantado como outro contêiner, o Serviço de Alertas (ADR-13).

5.6 Clean Architecture e Vertical Slice (reflete o código)

As duas abordagens se combinam: **Vertical Slice** define *como o código é agrupado* (uma pasta por funcionalidade) e **Clean Architecture** define *em que direção as dependências podem apontar* (sempre para o domínio). A Figura 4 mostra os pacotes e a regra de dependência.

```
api/src/
├── index.js                          registra as fatias (consumidor só com SERVICEBUS_CONSUMER_ENABLED=true)
├── bootstrap/container.js            Composition Root: escolhe adaptadores e injeta nos casos de uso
├── domain/                           NÚCLEO — sem dependências externas
│   ├── delivery/Delivery.js          entidade + regras de validação
│   ├── delivery/DeliveryStatus.js    value object
│   ├── delivery/DeliveryEvents.js    eventos de domínio (EDA)
│   ├── errors.js                     ValidationError, NotFoundError, InvalidIdentifierError
│   └── ports/deliveryPorts.js        portas: Writer, Reader, Updater, Remover, EventPublisher, AlertNotifier, Clock
├── features/                         FATIAS VERTICAIS (function + handler + caso de uso)
│   ├── createDelivery/   ├── searchDeliveries/   ├── updateDelivery/
│   ├── deleteDelivery/   ├── hello/              └── processDeliveryEvent/  (ServiceBusQueueTrigger)
├── infrastructure/                   ADAPTADORES
│   ├── persistence/mongo/            MongoConnection, MongoDeliveryRepository
│   ├── persistence/memory/           InMemoryDeliveryRepository
│   ├── messaging/                    ServiceBusDeliveryEventPublisher, LoggingEventPublisher, InMemoryEventPublisher
│   ├── notifications/                LogAlertNotifier
│   └── time/SystemClock.js
└── shared/http/                      httpResponse, requestReader, errorMapper
api/tests/
├── architecture/architecture.test.js 13 regras de arquitetura
└── unit/                             domínio, casos de uso, handlers, repositório e eventos
```

SOLID no código

Princípio	Evidência no código
S — Responsabilidade Única	Validação na entidade `Delivery`; orquestração no caso de uso; tradução HTTP no handler; persistência no repositório; registro de rota no `.function.js`.
O — Aberto/Fechado	Nova funcionalidade é uma nova pasta em `features/`, sem alterar as existentes. Novo erro é uma linha em `errorStatusTable`. Novo destino de eventos é um novo adaptador.
L — Substituição de Liskov	`MongoDeliveryRepository` e `InMemoryDeliveryRepository`, e os três publicadores de eventos, são intercambiáveis; `assertPort` verifica o contrato na construção.
I — Segregação de Interface	Portas pequenas: cada caso de uso recebe só o que usa (o Search recebe apenas `DeliveryReader`).
D — Inversão de Dependência	Casos de uso dependem de portas; apenas `bootstrap/container.js` instancia classes concretas (verificado por teste).

SEÇÃO 5.5 · C4 NÍVEL 3

C4 Nível 3 — Componentes do Serviço de Entregas

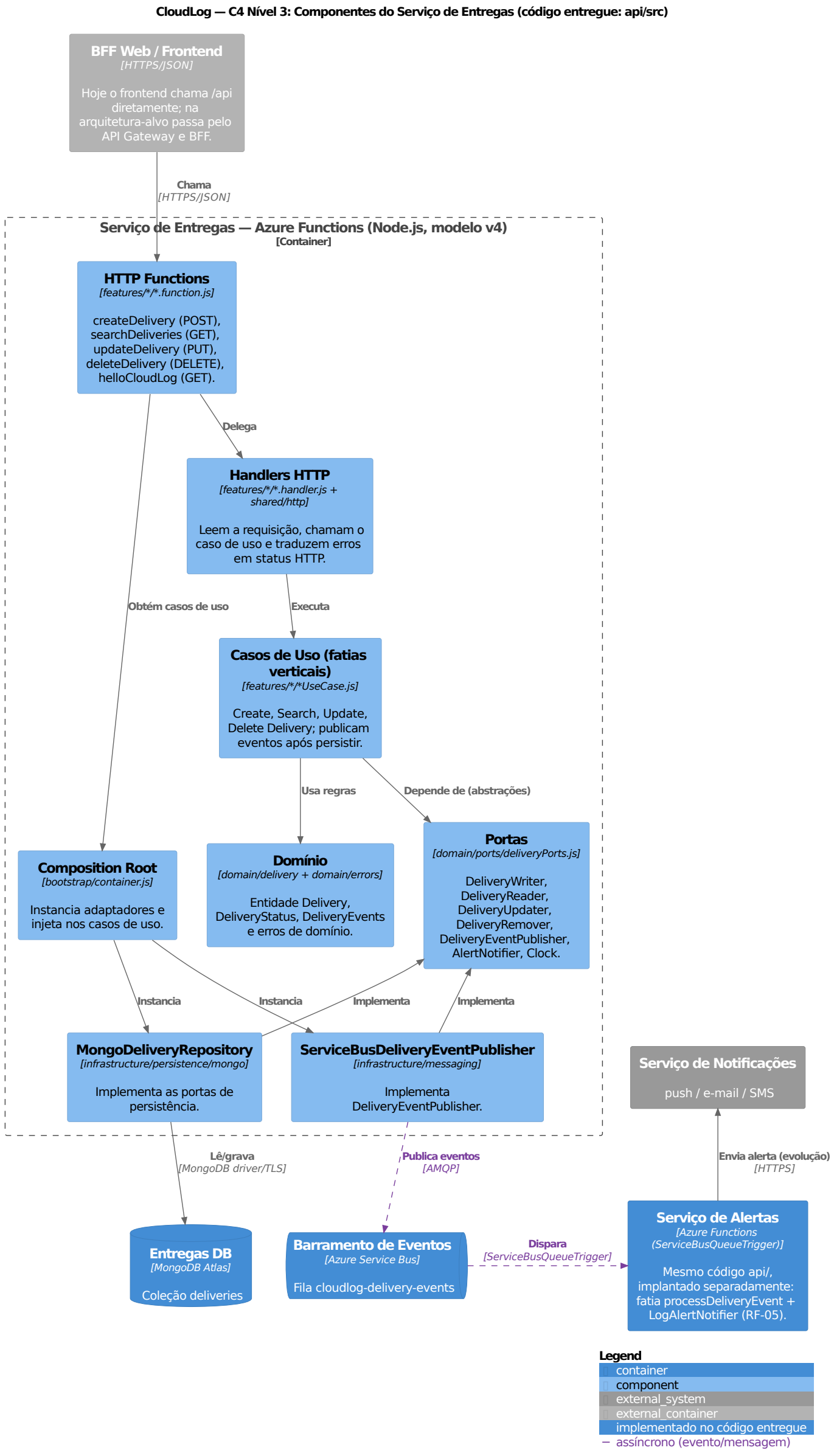


Figura 3 — Componentes do contêiner Serviço de Entregas, conforme o código em api/src. Fonte: `docs/arquitetura/diagramas/c4-3-componentes.puml`

SEÇÃO 5.6 · CLEAN ARCHITECTURE

Clean Architecture + Vertical Slice — Pacotes e regra de dependência

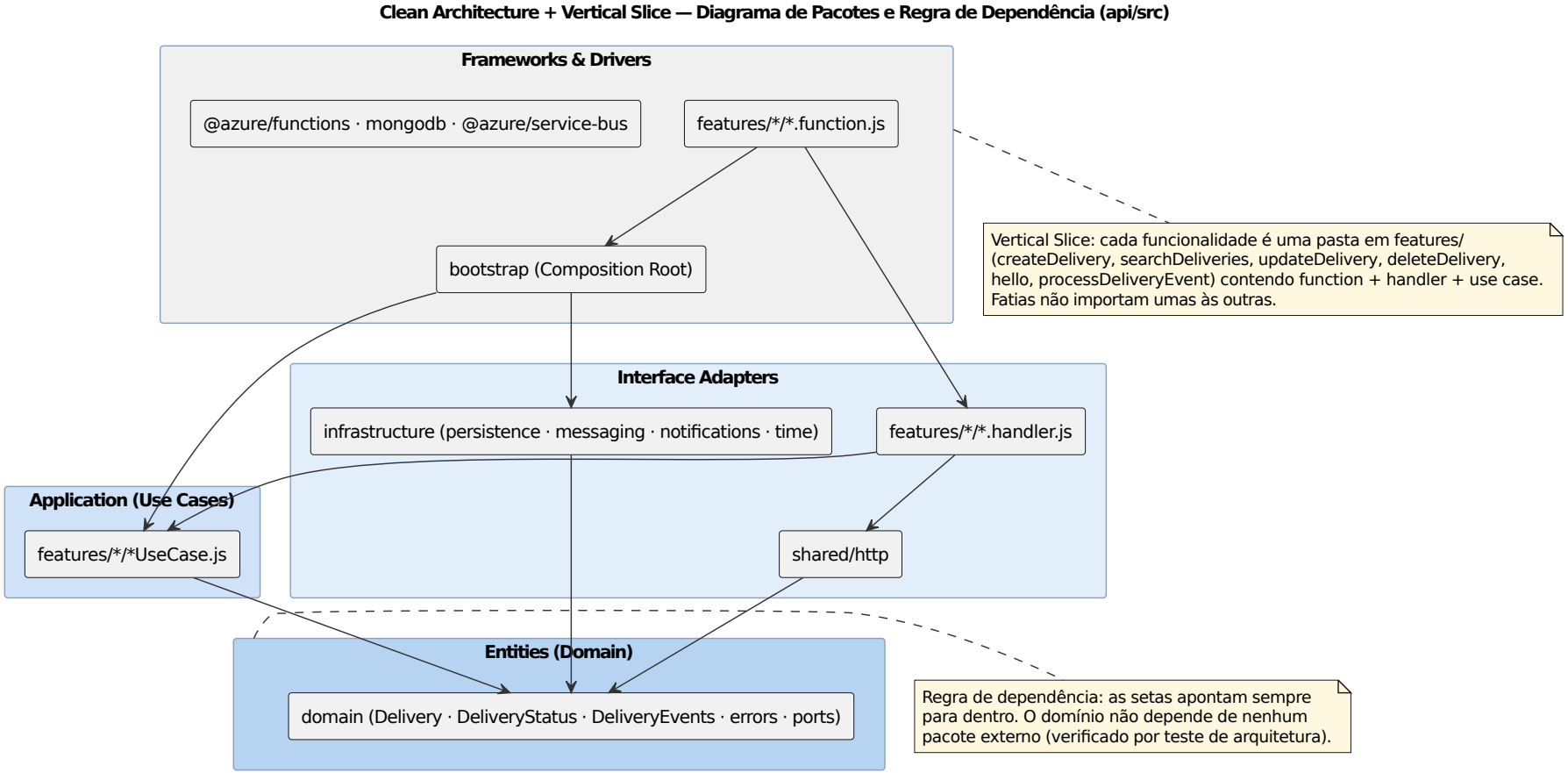


Figura 4 — Camadas da Clean Architecture aplicadas às fatias verticais do backend. Fonte: `docs/arquitetura/diagramas/uml-pacotes-clean.puml`

SEÇÃO 5

5.7 Nível 3 da decomposição — C4 Nível 4 (UML)

No nível de código usamos UML, como orienta a disciplina. Os diagramas foram desenhados a partir das classes reais do repositório:

- **Diagrama de Classes 1/2 (Figura 5)** — casos de uso de cada fatia, entidade `Delivery`, value object `DeliveryStatus`, evento de domínio `DeliveryEvent`, erros e portas que cada caso de uso exige.
- **Diagrama de Classes 2/2 (Figura 6)** — portas e os adaptadores que as realizam. Mostra a substituição (LSP): Mongo ou memória para persistência; Service Bus, log ou memória para eventos.
- **Diagrama de Componentes UML (Figura 7)** — interfaces fornecidas (círculo) e requeridas (seta tracejada «require») entre fatias, domínio e adaptadores.

5.8 Diagrama de Entidades e Relacionamentos

O DER (Figura 8) está organizado por banco de serviço (Database per Service). Relacionamentos com chave estrangeira existem só dentro do mesmo banco. Entre serviços há apenas **referência lógica por identificador** (linha tracejada), sem integridade referencial no banco, que é mantida pelos eventos. A entidade ENTREGA corresponde à coleção `deliveries`, que já está em produção no MongoDB Atlas; as demais entidades fazem parte da arquitetura-alvo.

Coleção implementada: `cloudlog.deliveries`

Campo	Tipo	Regra (validada na entidade Delivery)
_id	ObjectId	Gerado pelo MongoDB.
code, customer, destination, driver, eta	string	Obrigatórios na criação; espaços removidos.
status	string	Um de: Pendente, Em rota, Atrasada, Entregue.
progress	number	Opcional; entre 0 e 100.
createdAt, updatedAt	Date	Definidos pelo relógio (porta Clock).

SEÇÃO 5.7 · C4 NÍVEL 4 · UML CLASSES

C4 Nível 4 / UML — Diagrama de Classes 1/2

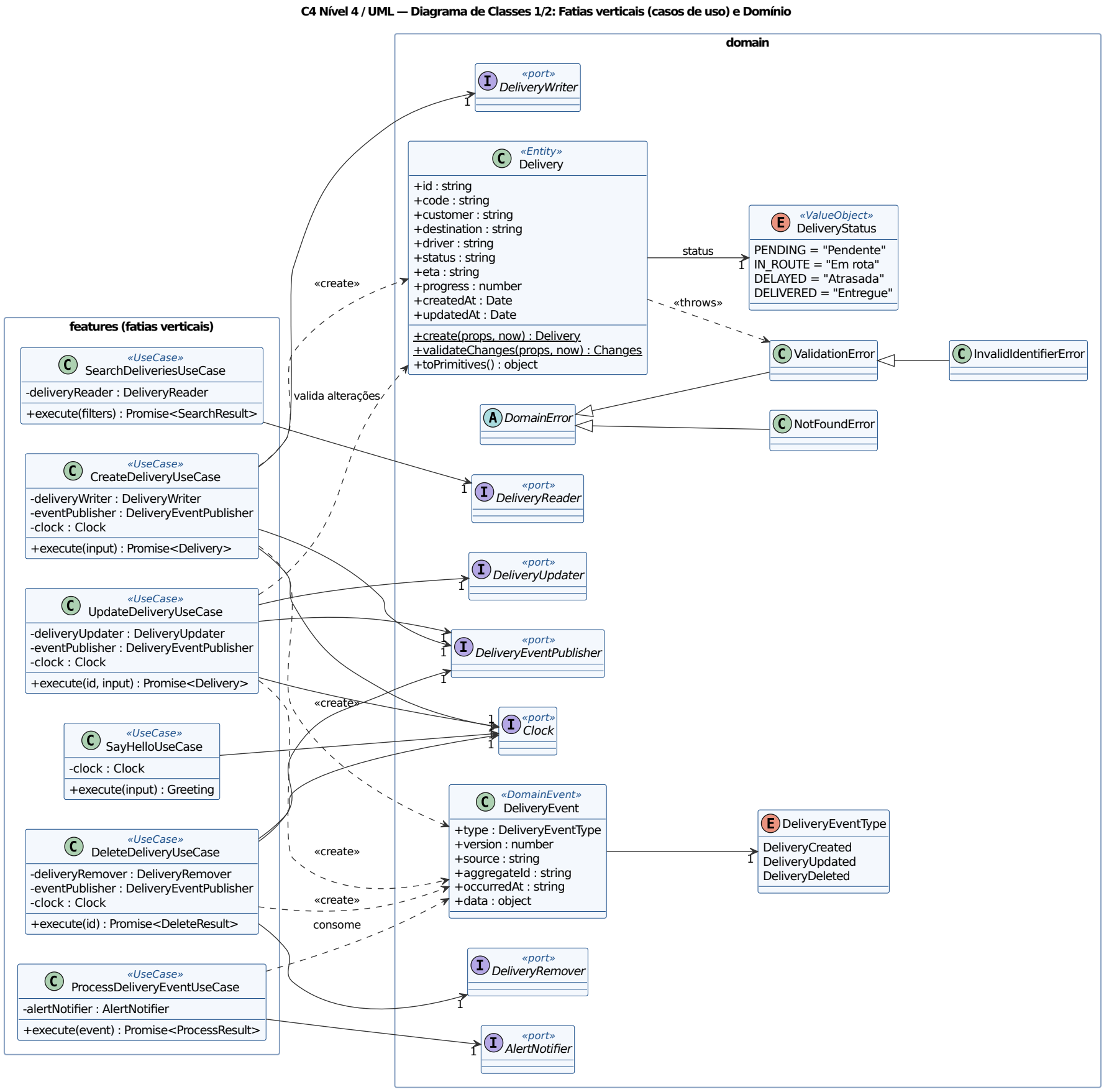


Figura 5 — Casos de uso das fatias verticais e o domínio. Fonte: `docs/arquitetura/diagramas/uml-classes-1-dominio.puml`

SEÇÃO 5.7 · C4 NÍVEL 4 · UML CLASSES

C4 Nível 4 / UML — Diagrama de Classes 2/2

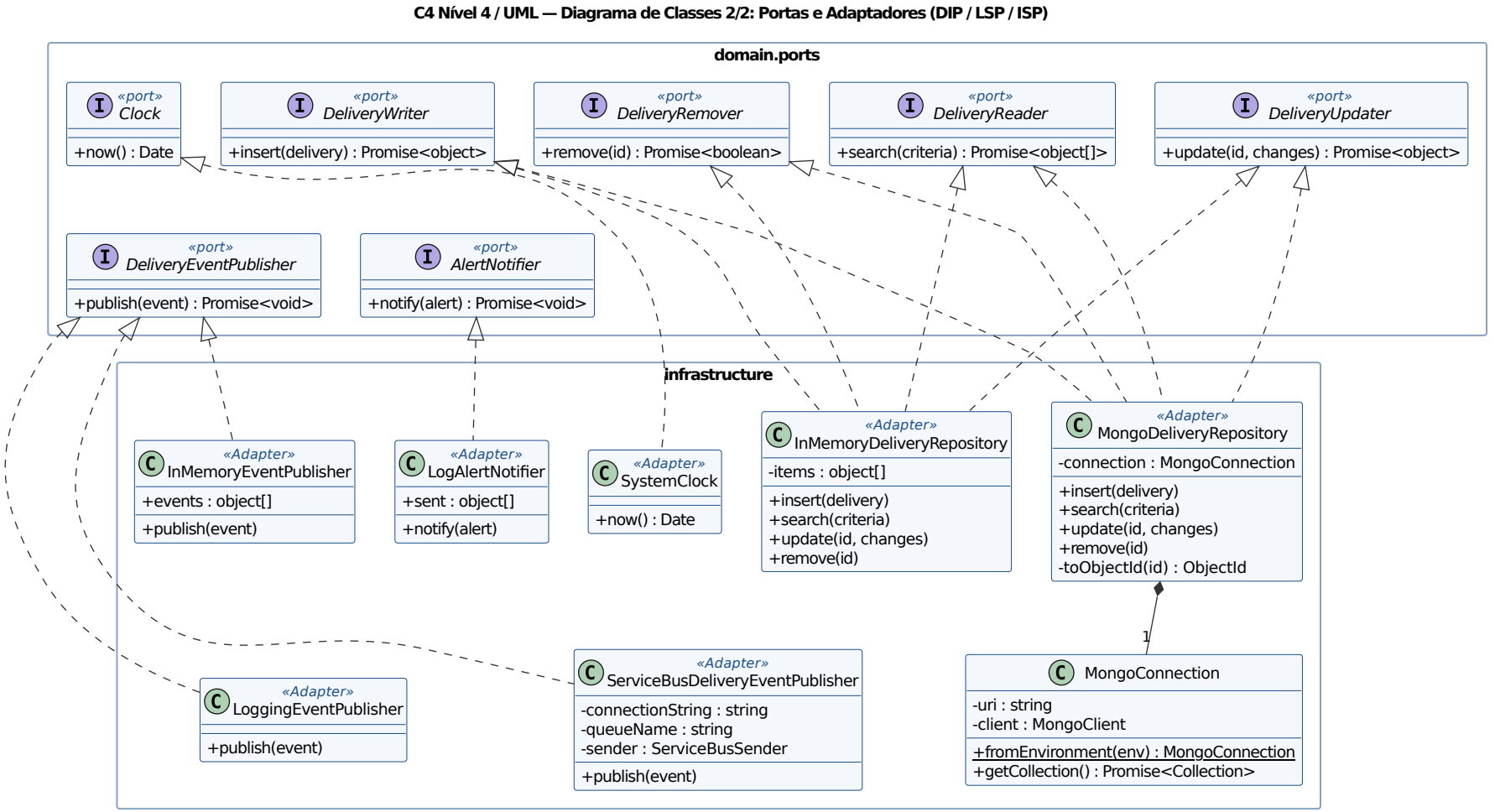


Figura 6 — Portas do domínio e adaptadores de infraestrutura que as realizam. Fonte: `docs/arquitetura/diagramas/uml-classes-2-adaptadores.puml`

SEÇÃO 5.7 · C4 NÍVEL 4 · UML COMPONENTES

C4 Nível 4 / UML — Diagrama de Componentes

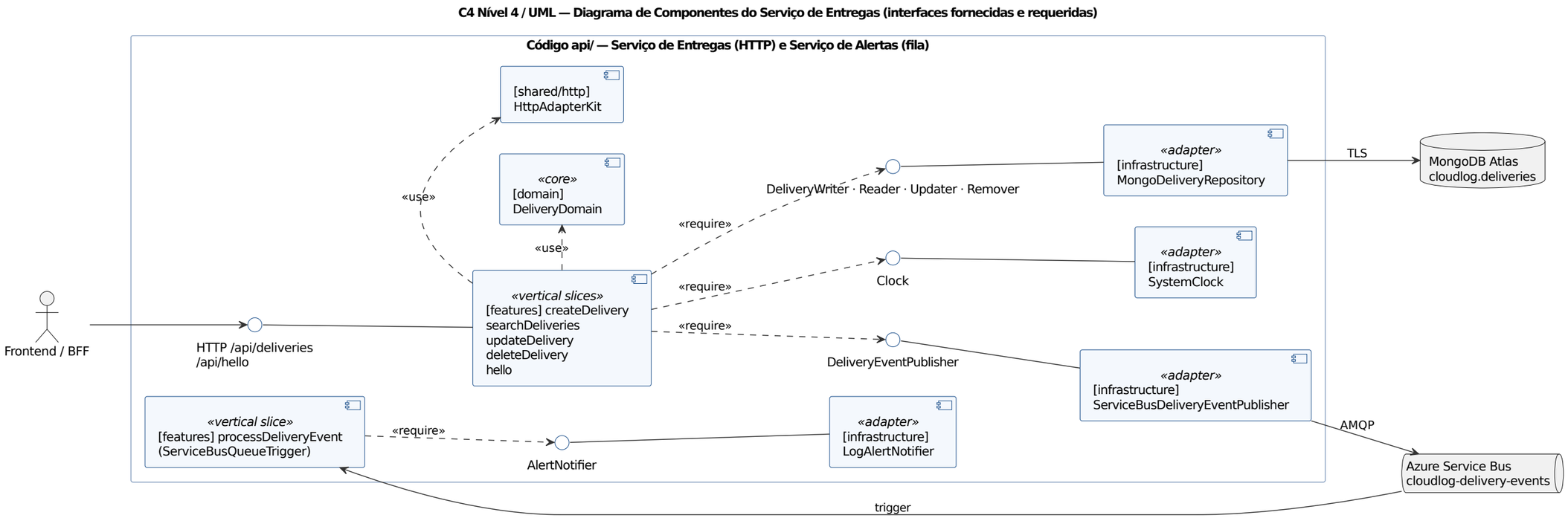


Figura 7 — Componentes do Serviço de Entregas com interfaces fornecidas e requeridas. Fonte: `docs/arquitetura/diagramas/uml-componentes.puml`

SEÇÃO 5.8 · DER

Diagrama de Entidades e Relacionamentos

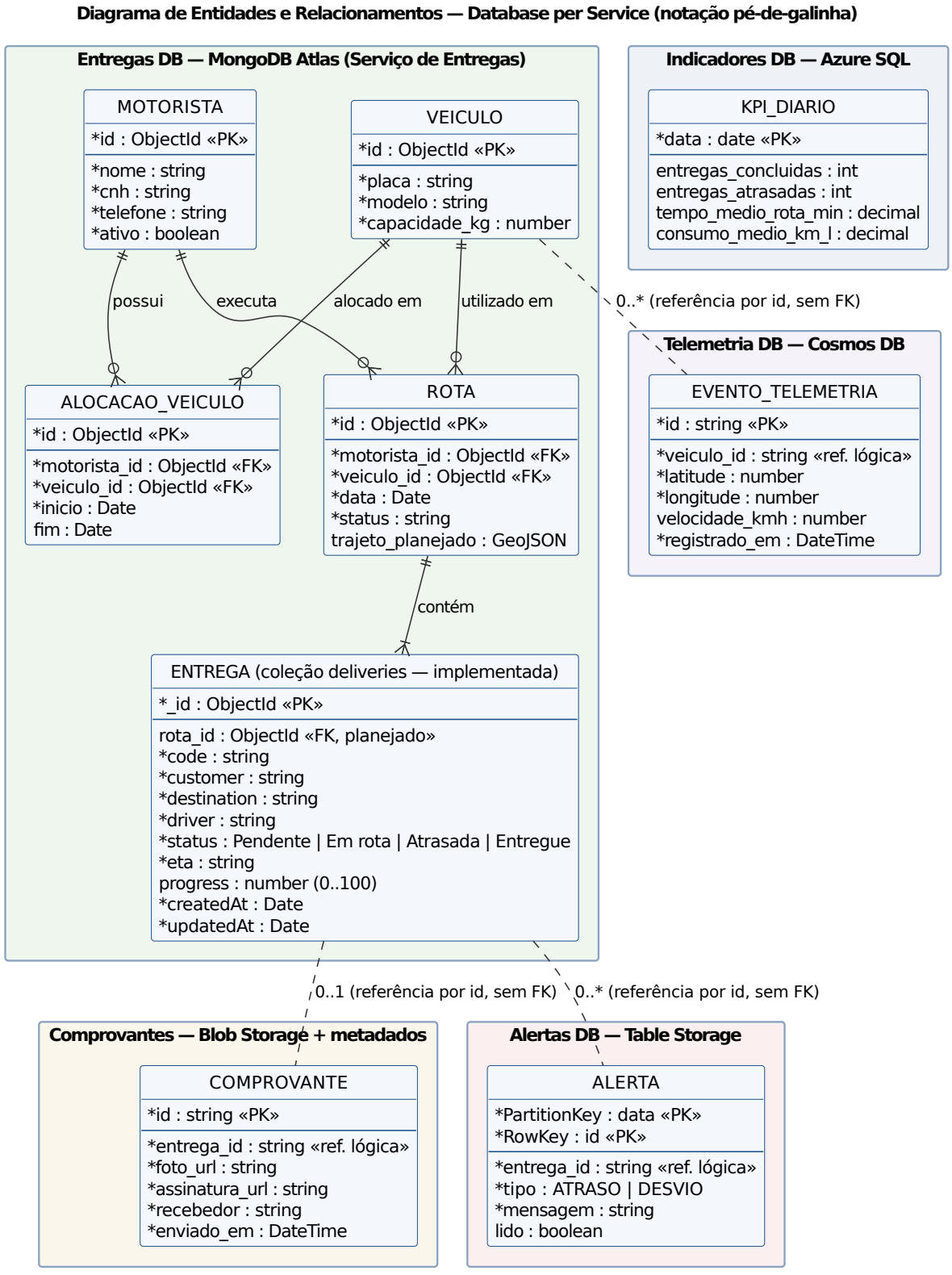


Figura 8 — Modelo de dados por serviço (Database per Service), em notação pé-de-galinha. Fonte: `docs/arquitetura/diagramas/der.puml`

SEÇÃO 6

6 Visão de Execução

> arc42: descreve o comportamento dinâmico dos blocos de construção em cenários concretos e importantes. No CloudLog, esses cenários são os diagramas de sequência UML do C4 Nível 4.

Cenário	Estilos envolvidos	Requisito	Situação
6.1 Inserir entrega com publicação de evento (Figura 9)	Serverless, Clean Architecture, Vertical Slice, EDA	RF-03	implementado
6.2 Entrega atrasada gera alerta por evento (Figura 10)	EDA, Serverless, Database per Service	RF-05	implementado
6.3 Rastreamento e desvio de rota (Figura 11)	EDA, Microsserviços, Serverless	RF-02, RF-05	planejado
6.4 Painel operacional via API Gateway e BFF (Figura 12)	Microfrontend, API Gateway, BFF, Microsserviços	RF-06	planejado

6.1 Inserir entrega

O handler lê o JSON e chama o caso de uso. A entidade valida os dados; se forem inválidos, o handler responde 400 sem tocar no banco. Se forem válidos, o repositório grava no MongoDB, o caso de uso publica `DeliveryCreated` e a API responde 201. O evento só é publicado depois da gravação, e nenhum evento sai quando a operação falha (coberto por teste).

6.2 Alerta de atraso por eventos

Quando uma entrega passa a *Atrasada*, a API responde ao usuário assim que grava e publica `DeliveryUpdated`. O alerta acontece depois, de forma assíncrona: o Service Bus aciona o consumidor `processDeliveryEvent`, que gera o alerta. Se o consumidor estiver fora do ar, a mensagem espera na fila; a API não é afetada.

6.3 Rastreamento e desvio de rota

As posições chegam por MQTT ao IoT Hub e disparam o Serviço de Telemetria, que grava no seu banco e compara com a rota planejada. Um desvio gera `RouteDeviationDetected`, consumido pelo Serviço de Alertas.

6.4 Painel operacional

O MFE Operação faz uma única chamada ao API Gateway. O BFF Web consulta Entregas, Telemetria e Indicadores em paralelo e devolve uma resposta pronta para a tela, evitando que o navegador faça várias chamadas.

6.5 Catálogo de eventos

Evento	Produtor	Consumidores	Quando	Situação
DeliveryCreated	Serviço de Entregas	Alertas, Indicadores	Após inserir uma entrega	implementado
DeliveryUpdated	Serviço de Entregas	Alertas (status Atrasada), Indicadores	Após alterar uma entrega; inclui `changedFields`	implementado
DeliveryDeleted	Serviço de Entregas	Indicadores	Após excluir uma entrega	implementado
PositionReceived	Telemetria	Indicadores	A cada posição recebida	planejado
RouteDeviationDetected	Telemetria	Alertas	Veículo fora do corredor da rota	planejado
ProofSubmitted	Comprovantes	Entregas (conclui), Indicadores	Após upload de foto e assinatura	planejado

Envelope dos eventos (DeliveryEvents.js)

```
{
  "type": "DeliveryUpdated",          // DeliveryEventType
  "version": 1,                       // versão do contrato do evento
  "source": "cloudlog/delivery-service",
  "aggregateId": "6720f1c2a9e4b1d3c0f5a911",
  "occurredAt": "2026-10-14T13:05:00.000Z",
  "data": { "code": "CL-2050", "status": "Atrasada", "driver": "Marcos Silva", ..., "changedFields": ["status", "updatedAt"] }
}
```

SEÇÃO 6.1 · C4 NÍVEL 4 · UML SEQUÊNCIA

C4 Nível 4 / UML — Sequência 1: Inserir entrega

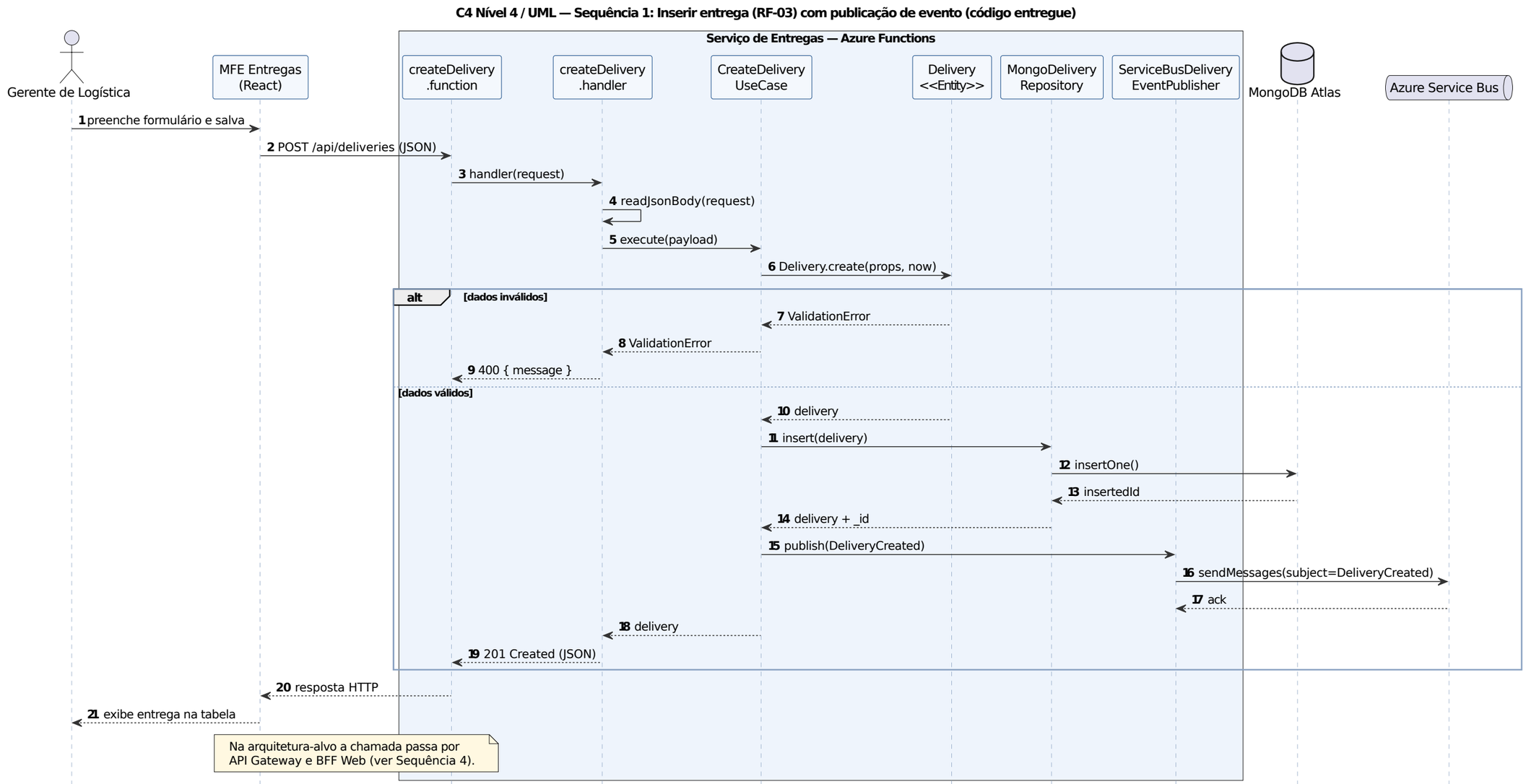


Figura 9 — Inserção de entrega com validação no domínio, persistência e publicação de DeliveryCreated. Fonte: `docs/arquitetura/diagramas/uml-seq-1-criar.puml`

SEÇÃO 6.2 · C4 NÍVEL 4 · UML SEQUÊNCIA

C4 Nível 4 / UML — Sequência 2: Alerta de atraso via eventos

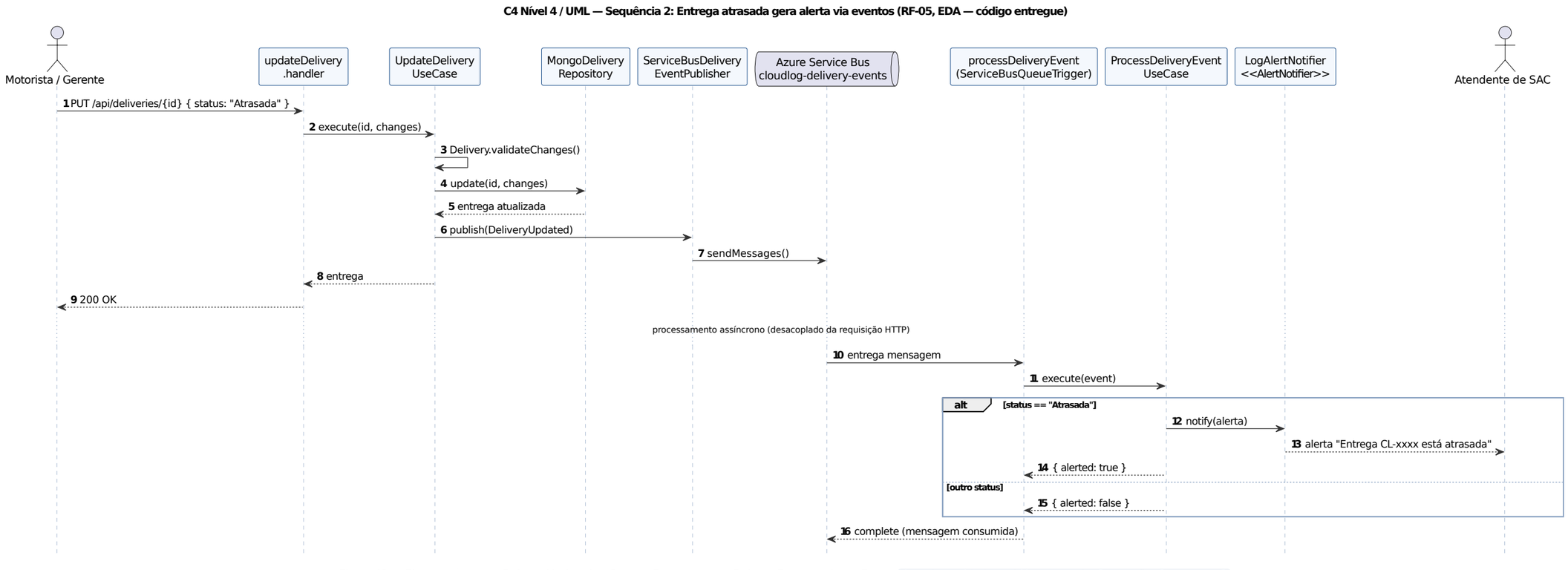


Figura 10 — Fluxo assíncrono: DeliveryUpdated → Service Bus → processDeliveryEvent → alerta. Fonte: `docs/arquitetura/diagramas/uml-seq-2-alerta.puml`

SEÇÃO 6.3 · C4 NÍVEL 4 · UML SEQUÊNCIA

C4 Nível 4 / UML — Sequência 3: Rastreamento e desvio de rota

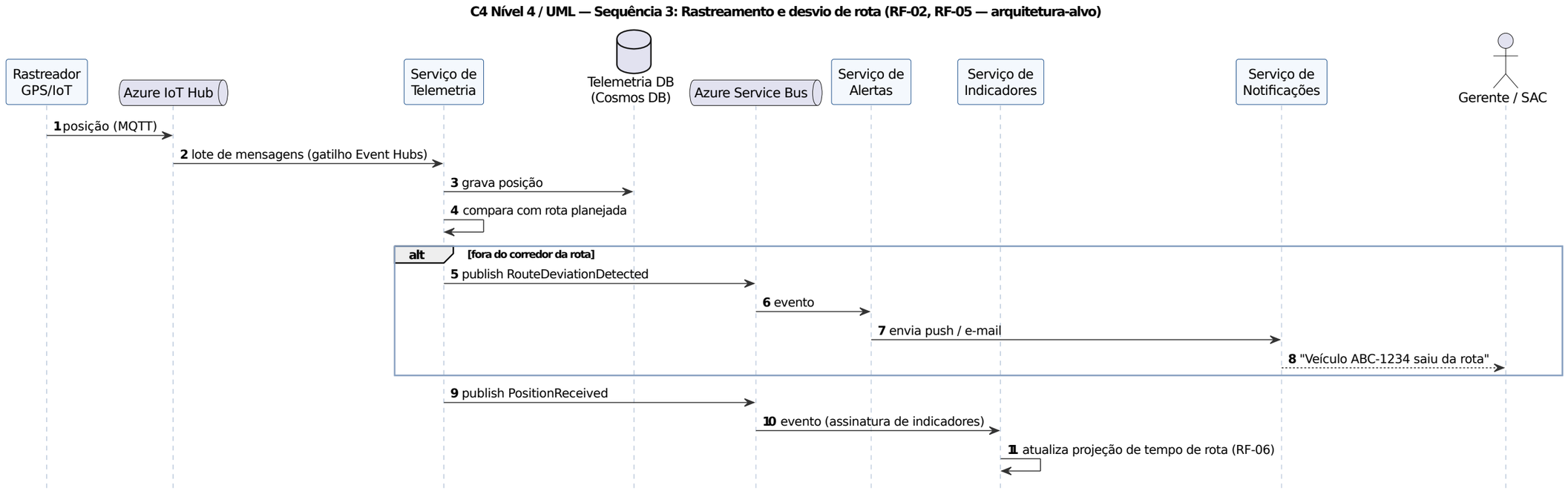


Figura 11 — Ingestão de telemetria e alerta de desvio (arquitetura-alvo). Fonte: `docs/arquitetura/diagramas/uml-seq-3-telemetria.puml`

SEÇÃO 6.4 · C4 NÍVEL 4 · UML SEQUÊNCIA

C4 Nível 4 / UML — Sequência 4: Painel via API Gateway e BFF

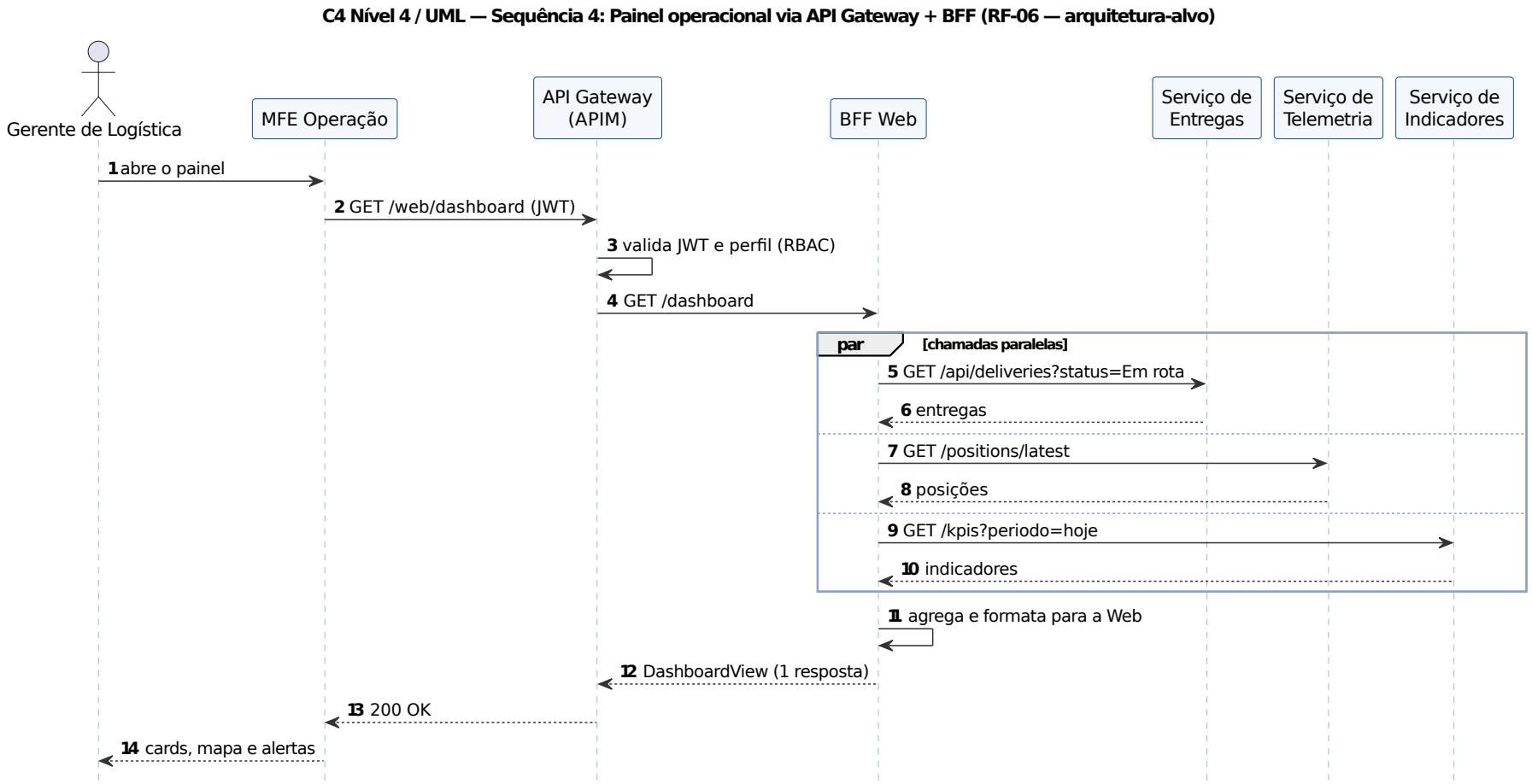


Figura 12 — Agregação de três microsserviços pelo BFF Web (arquitetura-alvo). Fonte: `docs/arquitetura/diagramas/uml-seq-4-bff.puml`

SEÇÃO 7

7 Visão de Implantação

> arc42: descreve a infraestrutura técnica em que o sistema executa e o mapeamento dos blocos de construção para essa infraestrutura.

Na versão parcial a implantação era genérica. Agora ela usa Microsoft Azure com serviços **serverless** e gerenciados, escolhidos pelo custo zero ou de consumo na assinatura de estudante (Figura 13).

Nó / recurso	Serviço Azure / externo	O que executa	Situação
Hospedagem web + API	Azure Static Web Apps (polite-mushroom-0b05fa610)	Frontend React e funções gerenciadas do Serviço de Entregas (`/api/*`)	implementado
Banco do Serviço de Entregas	MongoDB Atlas, cluster M0 *arquiteturacloud*	Banco `cloudlog`, coleção `deliveries`	implementado
Barramento de eventos	Azure Service Bus, namespace Basic, fila `cloudlog-delivery-events`	Transporte dos eventos de domínio	parcial configurável
Consumidor de eventos	Function App, plano Consumption	Mesmo código `api/` com `SERVICEBUS_CONSUMER_ENABLED=true`	parcial configurável
CI/CD	GitHub Actions	Testes unitários e de arquitetura em todo push; deploy no push da main	implementado
Observabilidade	Application Insights	Logs, métricas e erros das funções	parcial
Borda	API Management (Consumption)	API Gateway	planejado
BFFs	Function Apps (Consumption)	BFF Web e BFF Mobile	planejado
Dados dos demais serviços	Cosmos DB, Blob Storage, Table Storage, Azure SQL	Telemetria, comprovantes, alertas e indicadores	planejado
Ingestão IoT	Azure IoT Hub	Recepção MQTT dos rastreadores	planejado

7.1 Configuração por ambiente

Segredos não ficam no repositório: são configurações da plataforma, lidas pelo Composition Root.

Configuração	Onde	Uso
MONGODB_ATLAS_URI / _DATABASE / _COLLECTION	Static Web Apps → Variáveis de ambiente	Conexão do Serviço de Entregas com o MongoDB Atlas.
SERVICEBUS_CONNECTION, SERVICEBUS_QUEUE	Static Web Apps e Function App consumidora	Com conexão definida, eventos vão ao Service Bus; sem ela, apenas para o log.
SERVICEBUS_CONSUMER_ENABLED=true	Function App consumidora	Registra o gatilho de fila (o Static Web Apps aceita só HTTP).
DELIVERY_REPOSITORY=memory, EVENT_PUBLISHER=memory	Execução local e testes	Executa sem banco e sem Service Bus.

7.2 Pipeline de entrega

1. Push ou Pull Request alterando `api/` dispara o workflow *Backend — testes unitários e de arquitetura*.
2. Merge na `main` dispara o workflow do Azure Static Web Apps, que gera o build do frontend e publica frontend e API juntos.
3. A Function App consumidora recebe o mesmo pacote de `api/` com a variável que habilita o gatilho de fila.

SEÇÃO 7 · IMPLANTAÇÃO

Visão de Implantação — UML Deployment

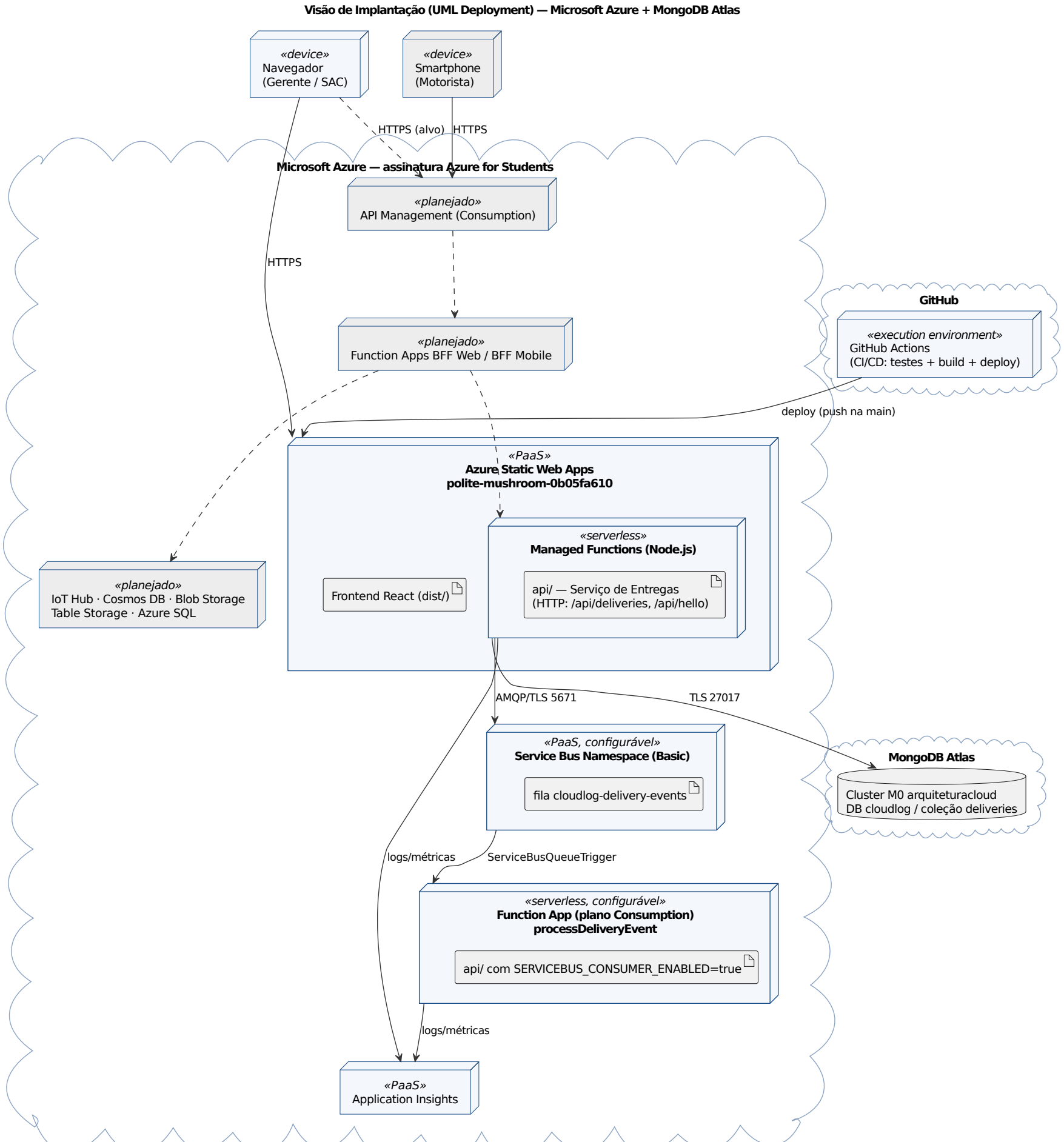


Figura 13 — Infraestrutura no Azure e no MongoDB Atlas. Nós cinza: arquitetura-alvo. Fonte: `docs/arquitetura/diagramas/implantacao.puml`

SEÇÃO 8

8 Conceitos Transversais

> arc42: regras e soluções que valem para várias partes do sistema, como segurança, tratamento de erros, integração, persistência e testes.

8.1 Segurança

Tema	Diretriz	Situação
Autenticação (RF-01)	Entra External ID com MFA; o Shell obtém o token por OIDC e o API Gateway valida o JWT.	planejado
Autorização	RBAC por perfil (Gerente, SAC, Motorista) aplicado no gateway e reforçado nos BFFs.	planejado
Segredos	Strings de conexão em configurações da plataforma; `local.settings.json` e `.env.local` fora do Git.	implementado
Transporte	HTTPS no Static Web Apps, TLS para MongoDB Atlas e AMQP/TLS para Service Bus.	implementado
Entrada de dados	Validação na entidade; regex de busca escapada para evitar injeção de operadores no MongoDB; respostas com `X-Content-Type-Options: nosniff`.	implementado
Arquivos (RF-04)	Blob privado com URL SAS de curta duração gerada pelo BFF Mobile.	planejado

8.2 Observabilidade

As funções registram logs no Application Insights. Erros inesperados são registrados com `context.error` antes de devolver o 500. Publicação de eventos e alertas geram linhas de log com o tipo do evento e o id da entrega, permitindo correlacionar uma requisição HTTP com o alerta gerado depois.

8.3 Tratamento de erros

O domínio lança erros sem conhecer HTTP. A tradução para status fica numa tabela única (`errorMapper.js`), aberta para extensão.

Erro	Origem	Resposta HTTP
InvalidRequestError (JSON inválido)	shared/http	400
ValidationError / InvalidIdentifierError	domain / adaptador Mongo	400 com mensagem amigável
NotFoundError	casos de uso	404
Qualquer outro erro	infraestrutura	500 genérico; detalhe só no log (`context.error`)

8.4 Integração por eventos (EDA)

Regra	Como é tratada
Eventos são fatos imutáveis	Criados por `DeliveryEvents` com `Object.freeze`, depois que a gravação deu certo.
Contrato versionado	Campo `version` no envelope; novas versões convivem com as antigas.
Payload serializável	Ids e datas convertidos para texto antes de publicar.
Publicação best-effort	Falha no Service Bus é registrada e não desfaz a operação (ADR-12). Evolução: Transactional Outbox.
Consumidor tolerante	Aceita mensagem como objeto ou JSON; evento sem `type` / `aggregateId` é rejeitado e, após as tentativas do Service Bus, vai para a dead-letter queue.
Idempotência	O alerta usa o id da entrega e o momento do evento; evolução: registrar ids processados no Alertas DB.

8.5 Serverless

Característica serverless	Tratamento no código
Funções sem estado	Nenhum estado em memória entre requisições, exceto conexões.
Reuso de conexão	`MongoConnection` e o sender do Service Bus são criados uma vez por instância e reaproveitados.
Cold start	Dependências leves e inicialização preguiçosa (conexão só no primeiro uso).
Gatilhos diferentes	O mesmo pacote atende HTTP (Static Web Apps) e fila (Function App), escolhido por configuração.

SEÇÃO 8

8.6 Testes unitários e testes de arquitetura (reflete o código)

O backend tem **40 testes** automatizados com o executor nativo do Node.js (`node:test`), sem dependências extras: **13 testes de arquitetura** e **27 testes unitários**. Eles rodam com `npm test` e no GitHub Actions.

Os testes de arquitetura funcionam no estilo do ArchUnit: o arquivo `tests/support/dependencyGraph.js` lê todos os arquivos de `api/src`, extrai cada `require(...)` e monta um grafo de dependências. Cada teste percorre o grafo e falha listando as violações encontradas (por exemplo, `features/createDelivery/CreateDeliveryUseCase.js -> mongodb`). Durante o desenvolvimento, introduzimos violações de propósito para conferir que os testes as detectam.

Regras de arquitetura verificadas

#	Estilo protegido	Teste (nome no código)	Por que importa
1	Clean Architecture	Domínio é independente: só depende de si mesmo (nenhum pacote, framework ou camada externa)	Garante o núcleo puro: nada em `domain/` importa pacotes ou outras camadas.
2	Clean Architecture	Casos de uso dependem apenas do domínio (sem HTTP, Azure ou MongoDB)	Use cases só conhecem domínio e portas.
3	Vertical Slice	Fatias verticais não dependem umas das outras	Uma fatia nunca importa arquivos de outra.
4	Clean Architecture	Infraestrutura não conhece features, HTTP nem bootstrap	Dependências da infraestrutura apontam para dentro.
5	Clean Architecture	shared/http não depende de features, infraestrutura ou bootstrap	Helpers HTTP são transversais e não dependem de fatias.
6	DIP / Database per Service	Pacote 'mongodb' só é usado no adaptador de persistência Mongo	Driver do banco confinado ao adaptador.
7	Serverless	Pacote '@azure/functions' só é usado nos arquivos *.function.js	Framework de funções confinado ao registro das rotas.
8	EDA	Pacote '@azure/service-bus' só é usado no adaptador de mensageria	SDK de mensageria confinado ao adaptador.
9	EDA	Casos de uso que alteram estado dependem da porta DeliveryEventPublisher (EDA)	Toda alteração de estado publica evento.
10	DIP	Somente o Composition Root (bootstrap) instancia a infraestrutura	Só o Composition Root cria classes concretas.
11	Governança	Nenhum pacote externo além dos declarados no package.json	Nenhuma dependência fora do package.json.
12	Vertical Slice	Cada fatia vertical tem UseCase, handler e function	Estrutura mínima de cada fatia.
13	Vertical Slice	index.js registra todas as fatias verticais	Toda fatia está registrada no ponto de entrada.

Testes unitários

Arquivo	O que cobre	Testes
tests/unit/domain.test.js	Regras da entidade Delivery e do DeliveryStatus	7
tests/unit/useCases.test.js	Casos de uso com repositório e publicador em memória (LSP)	7
tests/unit/handlers.test.js	Contrato HTTP usado pelo frontend, erros 400/404/500	5
tests/unit/mongoRepository.test.js	Adaptador Mongo: id inválido e filtro de busca	2
tests/unit/events.test.js	EDA: ordem dos eventos, nenhum evento em falha, consumidor de alertas, publicador Service Bus	6

```
$ cd api && npm test
# tests 40
# pass 40
# fail 0
```

SEÇÃO 9

9 Decisões Arquiteturais

> arc42: registra as decisões importantes, caras ou arriscadas, com motivo e consequências (no estilo ADR).

As decisões da versão parcial foram revistas. Três delas foram substituídas porque a implementação na disciplina definiu o provedor (Azure), o banco do Serviço de Entregas (MongoDB Atlas) e a decomposição em microsserviços.

ID	Decisão	Motivo	Relação com a v0.1	Consequências
ADR-01	Separar Web e Mobile	Perfis e contextos diferentes (escritório × estrada).	Mantida da v0.1.	Dois clientes; experiência adequada a cada perfil.
ADR-02	Microsserviços por capacidade, com Database per Service	Carga e evolução diferentes (telemetria × entregas); autonomia de cada parte.	Substitui a API central única da v0.1.	Consistência eventual entre serviços; mais componentes para operar.
ADR-03	Serverless com Azure Functions (Consumption)	Custo zero ocioso, escala automática e conta de estudante.	Substitui o 'provedor não definido' da v0.1.	Cold start; limites de tempo de execução.
ADR-04	MongoDB Atlas para o Serviço de Entregas	Entrega é um documento autocontido; camada gratuita no Student Pack; já usado na Entrega 3.	Substitui 'banco relacional' da v0.1 para este serviço.	Integridade referencial na aplicação; Azure SQL fica para Indicadores.
ADR-05	Object Storage para comprovantes	Custo e ciclo de vida de arquivos binários.	Mantida.	URLs SAS e metadados separados.
ADR-06	Integração por eventos no Azure Service Bus	Alertas e indicadores não devem atrasar nem derrubar o CRUD.	Evolução da v0.1 (alertas assíncronos).	Necessidade de idempotência e monitorar dead-letter.
ADR-07	API Gateway (Azure API Management)	Uma porta única para segurança, limites e roteamento.	Nova.	Ponto crítico de disponibilidade; custo por chamada.
ADR-08	Um BFF por canal (Web e Mobile)	Telas diferentes precisam de agregações diferentes; evita lógica de tela nos serviços.	Nova.	Mais um contêiner por canal; risco de duplicação entre BFFs.
ADR-09	Microfrontends com Module Federation	Telas por domínio evoluem e publicam separadamente.	Nova.	Dependências compartilhadas (React) precisam de versão alinhada.
ADR-10	Clean Architecture + Vertical Slice no backend	Mudanças localizadas por funcionalidade e domínio testável sem infraestrutura.	Nova (PR #1).	Mais arquivos por funcionalidade; exige disciplina mantida por testes.
ADR-11	Testes de arquitetura com node:test no CI	Proteger as regras das ADR-10 e ADR-06 sem adicionar bibliotecas.	Nova.	Regras baseadas em `require`; precisam ser atualizadas com novas pastas.
ADR-12	Publicação de eventos best-effort (sem outbox)	Simplicidade nesta fase.	Nova.	Um evento pode se perder se o Service Bus falhar após a gravação (R-01).
ADR-13	Consumidor da fila em Function App dedicada	Static Web Apps aceita apenas gatilhos HTTP.	Nova.	Mesmo código com duas implantações, controladas por variável.

SEÇÃO 10

10 Requisitos de Qualidade

> arc42: árvore de qualidade e cenários concretos e mensuráveis que tornam os objetivos de qualidade verificáveis.

10.1 Árvore de qualidade e cenários

Qualidade	Refinamento	Cenário	Medida / evidência
Disponibilidade	Falha de dependência	O Service Bus fica indisponível durante uma inserção.	A API responde 201 e registra a falha (teste *Publicador Service Bus ... não derruba a operação*).
Disponibilidade	Falha do consumidor	O consumidor de alertas cai por 10 min.	Mensagens ficam na fila e são processadas na volta; zero erro na API.
Desempenho	Latência	Gerente pesquisa entregas em horário de pico.	95% das respostas em até 800 ms (até 100 itens por consulta).
Desempenho	Telemetria	Posição válida recebida do rastreador.	Disponível no painel em até 10 s (95%).
Segurança	Autorização	Motorista tenta excluir uma entrega.	API Gateway responde 403 e registra o evento.
Escalabilidade	Carga variável	Número de veículos dobra.	Telemetria escala sozinha, sem alterar o Serviço de Entregas.
Manutenibilidade	Nova funcionalidade	Criar a funcionalidade “atribuir entrega a motorista”.	Nova pasta em `features/`; nenhuma fatia existente alterada; testes de arquitetura verdes.
Manutenibilidade	Troca de tecnologia	Trocar MongoDB por Cosmos DB.	Somente um novo adaptador e uma linha no Composition Root.
Testabilidade	Isolamento	Testar casos de uso sem banco nem Azure.	27 testes unitários usam adaptadores em memória.
Usabilidade	Comprovação	Motorista registra comprovante.	Até 5 interações após abrir a entrega.

10.2 Rastreabilidade entre requisitos, arquitetura e código

Requisito	Qualidades	Elementos da arquitetura	Evidência no código
RF-01	Segurança	Provedor de Identidade, API Gateway, BFFs	— (planejado)
RF-02	Desempenho, Escalabilidade	IoT Hub, Serviço de Telemetria, Cosmos DB	— (planejado)
RF-03	Disponibilidade, Manutenibilidade	Serviço de Entregas, MongoDB Atlas, MFE Entregas	Fatias create/search/update/deleteDelivery e seus testes
RF-04	Segurança, Usabilidade	App do Motorista, BFF Mobile, Serviço de Comprovantes, Blob	— (planejado)
RF-05	Disponibilidade	Service Bus, Serviço de Alertas	DeliveryEvents, fatia processDeliveryEvent, testes de eventos
RF-06	Desempenho	BFF Web, Serviço de Indicadores, MFE Operação	Tela Operação (dados de demonstração)

SEÇÃO 11

11 Riscos e Débitos Técnicos

> arc42: lista os riscos técnicos conhecidos e os débitos técnicos, ordenados por prioridade, com a mitigação prevista.

ID	Risco ou débito técnico	Impacto	Mitigação / próximo passo
R-01	Evento perdido se o Service Bus falhar logo após a gravação (sem outbox).	Alto	Transactional Outbox no MongoDB ou Change Streams publicando os eventos.
R-02	Rotas da API ainda sem autenticação (`authLevel: anonymous`).	Alto	Implantar API Gateway + Entra External ID (RF-01) antes de dados reais.
R-03	Cluster MongoDB Atlas M0 pausa por inatividade (ocorreu em outubro/2026 e gerou erro 500).	Médio	Monitorar com alerta de disponibilidade; tier pago em produção.
R-04	Cold start do plano Consumption aumenta a primeira resposta.	Médio	Flex Consumption ou instâncias pré-aquecidas nos serviços críticos.
R-05	Complexidade de operar vários microsserviços com equipe pequena.	Médio	Extrair serviços aos poucos, usando o Serviço de Entregas como modelo; IaC para os recursos.
R-06	Consistência eventual entre serviços pode confundir o usuário.	Médio	Mostrar estado “processando” e usar idempotência nos consumidores.
R-07	APIM, IoT Hub e Cosmos DB podem gerar custo fora da conta de estudante.	Médio	Usar camadas de consumo/gratuitas e excluir recursos após as avaliações.
D-01	Frontend ainda é uma SPA única (microfrontends planejados).	Baixo	Separar Shell e MFEs com Module Federation.
D-02	Consumidor sem registro de mensagens já processadas.	Baixo	Guardar ids processados no Alertas DB.
D-03	Sem testes de integração com MongoDB e Service Bus reais.	Médio	Testes de integração em pipeline com recursos de teste.
D-04	Alertas apenas registrados em log.	Baixo	Novo adaptador de AlertNotifier para push/e-mail (sem mudar o caso de uso).

SEÇÃO 12

12 Glossário

> arc42: termos de negócio e técnicos usados no documento.

Termo	Definição
ADR	Architecture Decision Record: registro curto de uma decisão de arquitetura.
API Gateway	Porta única de entrada das APIs, com segurança, limites e roteamento.
BFF	Backend for Frontend: backend específico de um canal (Web ou Mobile).
Composition Root	Único ponto do código que instancia implementações concretas e as injeta.
Database per Service	Cada microsserviço é dono exclusivo do seu armazenamento.
Dead-letter queue	Fila do Service Bus que recebe mensagens que falharam repetidamente.
EDA	Arquitetura Orientada a Eventos: serviços se comunicam publicando e consumindo eventos.
Evento de domínio	Fato de negócio já ocorrido, como DeliveryCreated.
Microfrontend	Parte do frontend com deploy independente, composta num Shell.
Module Federation	Recurso de bundlers (Webpack/Vite) para carregar módulos remotos em tempo de execução.
Porta / Adaptador	Porta é a interface exigida pelo núcleo; adaptador é a implementação na infraestrutura.
Serverless	Modelo em que a plataforma gerencia servidores e escala; cobrança por execução.
Teste de arquitetura	Teste automatizado que verifica regras de dependência e estrutura do código.
Vertical Slice	Organização do código por funcionalidade, de ponta a ponta, em vez de por camada técnica.